Nexus HMS Deployment Strategy

Nexus HMS Enterprise Edition

Document control

- Product: Nexus HMS Enterprise Hotel Management System
- Release baseline: Phase 7B Enterprise Platform
- Document type: Deployment and operations guide
- Date: 27 August 2026

1. Deployment principles

Deploy the API, web application, PostgreSQL database, Redis realtime backplane, and file storage as separate operational concerns. Keep environment secrets outside source control. Build once, promote the same verified artifact between environments, and run database changes before enabling code that depends on them.

The current repository includes managed development workflows for the API and web application. Docker and Kubernetes are recommended production packaging strategies, but their manifests should be treated as release engineering deliverables and reviewed against the current repository before use.

2. Environment model

- Development: local or Replit workspace, seeded data, verbose operational feedback, no production credentials.
- Staging: production-like database version, representative data shape, masked data, full smoke and authorization testing.
- Production: isolated database and secret scope, restricted network access, backups, monitoring, and controlled releases.

Do not point development or staging code at a production database. Use a separate ‘DATABASE_URL‘ for every environment.

3. Required configuration

Variable | Purpose | Guidance

DATABASE_URL | PostgreSQL connection | Use a private, TLS-capable production connection

SESSION_SECRET | JWT and MFA key material | Generate a long random value and store it in a secret manager

REDIS_PASSWORD | Included Redis service authentication | Required by Compose and Kubernetes Redis; store it as a secret

REDIS_URL | Redis 7 connection for the Socket.IO adapter | Required in production; include the matching credential and store the complete URL as a secret

PORT | API listen port | Use the platform-provided port; current development API defaults to 8080

NODE_ENV | Runtime behavior | Set ‘production‘ for production

NEXT_PUBLIC_API_URL | Browser API origin | Use the public API origin or same-origin proxy path

The application must start with ‘SESSION_SECRET‘ set because MFA encryption intentionally fails closed without it. A production API also fails startup when ‘REDIS_URL‘ is absent, because Redis is required for cross-replica Socket.IO delivery. For a real production environment, prefer a managed Redis service on a private network with TLS and use a credential-bearing ‘rediss://‘ URL. The included internal Redis deployment uses password authentication but does not enable TLS. CORS origins, ingress rules, and metrics access should be narrowed in production.

4. Local deployment

1. Install Node.js and pnpm versions supported by the workspace.

2. Run ‘pnpm install‘.

3. Configure ‘DATABASE_URL‘ and ‘SESSION_SECRET‘ in the local environment. When exercising the Redis-backed Socket.IO adapter, configure matching ‘REDIS_PASSWORD‘ and credential-bearing ‘REDIS_URL‘ values.

4. Build the shared database package with ‘pnpm run typecheck:libs‘ or the database package build command.

5. Apply the approved SQL migrations to the development database.

6. Run the seed scripts appropriate to the desired data set.

7. Start the API with ‘pnpm --filter @workspace/api-server run dev‘.

8. Start the web app with ‘pnpm --filter @workspace/hotel-management run dev‘.

9. Verify ‘GET /api/health‘, ‘GET /api/metrics‘, login, and a representative protected request.

The API health response should be machine-readable with status ‘ok‘. The metrics endpoint is intentionally unauthenticated; do not expose it publicly outside development.

5. Replit workflow deployment

Use the existing managed workflows rather than creating duplicate servers. Confirm the API workflow binds to the configured port and that the web workflow points to the intended API route. After code, package, or run-command changes:

1. Restart the affected workflow.

2. Check workflow and browser console logs.

3. Verify the health endpoint and the browser preview.

4. Run the changed user flow through a real browser smoke test.

Keep environment variables and secrets in the workspace’s environment configuration. Never commit ‘.env‘ files or paste secret values into tickets.

6. Container strategy

API image

Use a multi-stage Node image. The build stage installs workspace dependencies, compiles the shared database package, and builds the API. The runtime stage contains production dependencies and compiled output only. Run ‘node dist/main.js‘, expose the configured API port, and use a non-root user where the base image policy permits.

Web image

Build the Next.js application with the configured standalone output when enabled. The runtime image should contain the standalone server and static assets only. Expose the web port and pass the API origin through deployment configuration rather than baking a development URL into source.

Compose

The Compose stacks include PostgreSQL where appropriate, Redis 7, API, and web services with health checks, restart behavior, and a private service network. Redis starts with ‘--requirepass‘; ‘REDIS_PASSWORD‘ and the password in the API’s ‘REDIS_URL‘ must match. Redis health checks authenticate. Redis provides the Socket.IO backplane only: cache and EventEmitter2 remain process-local, and uploads remain on the local Compose volume. These internal Compose connections use ‘redis://‘, not TLS, and are not the recommended Redis topology for real production.

The API container smoke gate starts PostgreSQL 16 and password-protected Redis 7, authenticates the Redis readiness check, starts two production API containers with credential-bearing Redis URLs, checks ‘/api/health‘ on both, and verifies that uid 1000 can write to the shared mounted upload volume.

7. Kubernetes strategy

Use a namespace per environment or a clearly isolated release boundary. Recommended resources:

- API Deployment with three replicas and an HPA ranging from two to ten replicas.
- Web Deployment with at least two replicas for workstation availability.
- Redis 7 Deployment and ClusterIP Service for the Socket.IO adapter.
- ClusterIP Services for API and web.
- Ingress with TLS and an approved hostname.
- Readiness and liveness probes on ‘/api/health‘.
- CPU and memory requests and limits.
- HPA based on CPU plus application metrics where appropriate.
- Secret objects or an external secret provider for ‘DATABASE_URL‘ and ‘SESSION_SECRET‘.
- A 10Gi ReadWriteOnce uploads PVC.

The Redis Deployment requires ‘REDIS_PASSWORD‘, and the API reads its credential-bearing ‘REDIS_URL‘ from the Kubernetes Secret rather than the ConfigMap. A NetworkPolicy permits Redis ingress only from API pods on TCP 6379, and a second policy denies all Redis egress. Redis and web containers explicitly run as non-root, disallow privilege escalation, and drop all Linux capabilities; Redis also uses a read-only root filesystem.

The Redis adapter provides cross-replica Socket.IO delivery. The current API Deployment also has required pod affinity that co-locates all API pods on one node so they can mount the 10Gi ReadWriteOnce uploads PVC. This preserves the three-replica/HPA template within a single-node storage boundary, but it limits node-failure resilience and scheduling capacity. Multi-node API placement requires shared filesystem storage with a suitable access mode or object storage. The process-local cache and EventEmitter2 event bus do not synchronize state between replicas; Redis is neither their shared cache nor a durable EventBus.

8. Database migration and release order

1. Take or verify a database backup.
2. Apply additive migrations in staging.
3. Run schema and API builds.
4. Run authorization, MFA, mobile, metrics, and critical reservation smoke tests.
5. Apply the migration to production during the approved window.
6. Deploy the compatible API and web artifacts.
7. Verify health, metrics, login, MFA, and one representative workflow.
8. Monitor errors and latency before announcing completion.

Do not run destructive schema changes in the same release as an unverified application change. Prefer expand, backfill, verify, migrate, and contract sequencing.

9. Networking and security controls

- Terminate TLS at the approved edge or ingress.
- Allow database traffic only from the API runtime.
- Allow Redis traffic only from API pods and deny Redis egress.
- Restrict ‘/api/metrics‘ to the monitoring network.
- Configure CORS to approved web origins.
- Use network-level IP rules as a second layer; application IP restrictions are not a substitute for a firewall.
- Rotate credentials through the secret manager, with a documented token and session invalidation plan.
- Prefer managed Redis over the included internal deployment for real production, keep it private, require authentication and TLS, and use ‘rediss://‘. Do not claim TLS for the included Redis manifests.

10. Monitoring, backups, and rollback

Monitor health checks, request totals and duration, active WebSocket connections, cache hit/miss ratios, database pool pressure, application errors, and authentication failures. Back up PostgreSQL on a schedule aligned with the hotel recovery point objective and test restoration.

For rollback, stop promotion of the current artifact, deploy the last verified application image, and only roll back the database when the migration is explicitly reversible and the data impact is understood. If the database migration is additive, prefer a forward-compatible application rollback rather than destructive reversal.

11. Production checklist

- [ ] Production ‘SESSION_SECRET‘ is set and not reused from another environment.
- [ ] ‘REDIS_PASSWORD‘ and the credential-bearing ‘REDIS_URL‘ are consistent and stored as secrets.
- [ ] Real production Redis is managed/private with TLS and the API uses ‘rediss://‘, or an explicitly approved equivalent control is documented.
- [ ] Redis NetworkPolicies restrict ingress to API pods and deny Redis egress.
- [ ] Redis and web pods run non-root without privilege escalation.
- [ ] Database backups and restore owner are documented.
- [ ] CORS and metrics ingress are restricted.
- [ ] Health and metrics checks are monitored.
- [ ] At least one admin and one operational user have been verified.
- [ ] MFA enrollment and backup-code recovery have been tested.
- [ ] Cross-replica Socket.IO delivery through Redis has been verified.
- [ ] Required API pod affinity is retained while the ReadWriteOnce uploads PVC is used.
- [ ] File storage backup and retention are configured.
- [ ] Shared or object storage is implemented before API pods are distributed across nodes.
- [ ] Tenant isolation limitations are accepted by the release owner.
- [ ] Rollback artifact and migration plan are recorded.

12. Known deployment boundaries

Phase 7B uses Redis 7 for the implemented Socket.IO multi-replica backplane. It still deliberately uses an in-memory process-local cache, an in-process EventEmitter2 bus, PostgreSQL ILIKE search, local filesystem storage, and stored-but-not-delivered push tokens. The Kubernetes template keeps API replicas on one node for its 10Gi ReadWriteOnce upload volume; shared or object storage is required before multi-node or multi-region API operation.